Automatización de tareas

Contenidos

1 Automatización de tareas **1**
- 1.1 Airflow . . . 1
- 1.2 Microsoft Power Auomate . . . 1
- 1.3 Luigi . . . 1
- 1.4 Dagster . . . 1

2 Tarea **1**
- 2.1 Ejemplo Airflow . . . 2

1 Automatización de tareas

1.1 Airflow

Airflow es una plataforma de orquestación de flujo de trabajo que se utiliza para automatizar y programar tareas en un entorno de datos. Permite crear flujos de trabajo (DAG) de manera fácil y rápida, lo que puede ser muy útil en el contexto de machine learning.

1.2 Microsoft Power Auomate

1.3 Luigi

1.4 Dagster

2 Tarea

Un pequeño flujo de trabajo con Airflow relacionado con machine learning podría involucrar los siguientes pasos:

1. Extracción y preprocesamiento de datos: Crear un DAG que extraiga datos de una fuente externa, como una base de datos o un archivo CSV. Una vez que se extraen los datos, se pueden realizar operaciones de preprocesamiento para preparar los datos para el modelado. Esto podría incluir la limpieza de datos faltantes, la eliminación de variables innecesarias, la codificación de variables categóricas, y la normalización de variables numéricas.
2. Entrenamiento del modelo: Crear una tarea en el DAG que entrene un modelo de machine learning utilizando los datos preprocesados. Esto puede implicar la división de los datos

en conjuntos de entrenamiento y prueba, la selección de un algoritmo de machine learning adecuado y la optimización de sus parámetros.

3. Validación del modelo: Una vez que se ha entrenado el modelo, crear una tarea que realice la validación del modelo con los datos de prueba. Esto puede involucrar la evaluación del rendimiento del modelo utilizando métricas de evaluación, como el error cuadrático medio o la precisión.

4. Despliegue del modelo: Una vez que se ha validado el modelo, crear una tarea que implemente el modelo en un entorno de producción. Esto puede implicar el empaquetamiento del modelo en un formato que pueda ser utilizado por otros sistemas, como una API RESTful, o la integración del modelo en una plataforma de nube, como AWS o Google Cloud.

5. Monitoreo del modelo: Finalmente, crear una tarea que monitoree continuamente el rendimiento del modelo en producción, y alerte a los usuarios si el rendimiento del modelo disminuye o si se detectan problemas.

2.1 Ejemplo Airflow

1. Crear el archivo DAG Primero, crearemos un archivo Python que definirá nuestro DAG. Este archivo puede llamarse, por ejemplo, iris_ml_workflow.py. A continuación se muestra el código que debe contener:

```python
from datetime import datetime, timedelta
from airflow import DAG
from airflow.operators.python_operator import PythonOperator

# Importar las funciones que definiremos más adelante
from data_extraction import extract_data
from data_preprocessing import preprocess_data
from train_model import train_model
from validate_model import validate_model

default_args = {
    'owner': 'airflow',
    'depends_on_past': False,
    'start_date': datetime(2023, 4, 28),
    'email': ['airflow@example.com'],
    'email_on_failure': False,
    'email_on_retry': False,
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
}

dag = DAG(
    'iris_ml_workflow',
    default_args=default_args,
```

```
    description='A workflow to train and validate'
    'a machine learning model for Iris classification',
    schedule_interval=timedelta(days=1),
)

# Definir las tareas del DAG
extract_task = PythonOperator(task_id='extract_data',
                        python_callable=extract_data, dag=dag)
preprocess_task = PythonOperator(task_id='preprocess_data',
                        python_callable=preprocess_data, dag=dag)
train_task = PythonOperator(task_id='train_model',
                        python_callable=train_model, dag=dag)
validate_task = PythonOperator(task_id='validate_model',
                        python_callable=validate_model, dag=dag)

# Establecer las dependencias entre las tareas
preprocess_task.set_upstream(extract_task)
train_task.set_upstream(preprocess_task)
validate_task.set_upstream(train_task)
```

En este código, creamos un DAG con el nombre "iris_ml_workflow". Definimos cuatro tareas: "extract_data", "preprocess_data", "train_model" y "validate_model". También establecemos las dependencias entre las tareas.

1. Crear las funciones para las tareas

A continuación, crearemos las funciones que se ejecutarán en cada tarea. Suponemos que tienes los archivos data_extraction.py, data_preprocessing.py, train_model.py y validate_model.py en el mismo directorio que el archivo iris_ml_workflow.py. A continuación se muestra el código para cada archivo. data_extraction.py:

```
import pandas as pd
from sklearn.datasets import load_iris

def extract_data():
    iris = load_iris()
    data = pd.DataFrame(iris.data, columns=iris.feature_names)
    target = pd.DataFrame(iris.target, columns=['target'])
    data.to_csv('/path/to/data/iris_data.csv', index=False)
    target.to_csv('/path/to/data/iris_target.csv', index=False)
```

data_preprocessing.py:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
def preprocess_data():
    data = pd.read_csv('/path/to/data/iris_data.csv')
    target = pd.read_csv('/path/to/data/iris_target.csv')
    X_train, X_test, y_train, y_test =
             train_test_split(data, target, test_size=0.2)
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)
    X_train_scaled.to_csv('/path/to/data/iris_train_data.csv', index=False)
    y_train.to_csv('/path/to/data/iris_train_target.csv', index=False)
    X_test_scaled.to_csv('/path/to/data/iris_test_data.csv', index=False)
    y_test.to_csv('/path/to/data/iris_test_target.csv', index=False)
```

'train_model.py':

```
import pandas as pd
from sklearn.linear_model import LogisticRegression

def train_model():
    X_train = pd.read_csv('/path/to/data/iris_train_data.csv')
    y_train = pd.read_csv('/path/to/data/iris_train_target.csv')
    clf = LogisticRegression()
    clf.fit(X_train, y_train)
    joblib.dump(clf, '/path/to/models/iris_model.joblib')
```

validate_model.py:

```
import pandas as pd
from sklearn.metrics import accuracy_score
import joblib

def validate_model():
    X_test = pd.read_csv('/path/to/data/iris_test_data.csv')
    y_test = pd.read_csv('/path/to/data/iris_test_target.csv')
    clf = joblib.load('/path/to/models/iris_model.joblib')
    y_pred = clf.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    with open('/path/to/results/accuracy.txt', 'w') as f:
        f.write(str(accuracy))
```

En estas funciones, extraemos los datos del dataset Iris, los preprocesamos, entrenamos un modelo de regresión logística y validamos el modelo en un conjunto de prueba. Los datos y modelos se guardan en archivos en el sistema de archivos local.

3.Ejecutar el DAG Una vez que hayamos creado nuestro DAG y las funciones para las tareas, podemos iniciar Airflow y ejecutar el DAG. Para iniciar Airflow, ejecuta el siguiente comando en

tu terminal:

```
airflow webserver -p 8080
```

Luego, en otra terminal, ejecuta el siguiente comando:

```
airflow scheduler
```

Esto iniciará los servidores de Airflow y el planificador, respectivamente. Luego, en tu navegador, ve a localhost:8080 para acceder a la interfaz de usuario de Airflow. Allí, deberías ver el DAG "iris_ml_workflow" en la pantalla. Haz clic en "Trigger DAG" para iniciar el flujo de trabajo.

Si todo ha ido bien, deberías ver que cada tarea se ejecuta en secuencia. Los archivos de datos, modelos y resultados deberían estar disponibles en el sistema de archivos local, y deberías poder revisar el archivo accuracy.txt para ver la precisión del modelo en el conjunto de prueba.